

GOVT 6019: Homework 1 — Solutions

Fall 2026 · Data, Variables, Notation

Melissa Sands (Instructor)

Izza Malik (Teaching Assistant)

i How to read these solutions

Compare against your own written attempt, not your recollection of what you “would have said.” For the concept questions, several defensible answers exist; what matters is whether your justification survives contact with the ones below. All numbers in Part C are computed live from `voter2012.csv` — the code is shown so you can rerun every line.

Part A — Concepts

Question 1: Levels of measurement

- a. **pid3** — **nominal, discrete**. The codes 1/2/3 are arbitrary labels; there is no sense in which “Other” (3) is *more* than “Republican” (2). Any arithmetic on the codes — an average of 1.7, say — is meaningless.
- b. **educ** — **ordinal, discrete**. The six categories are genuinely ordered (post-grad is more education than no HS), but the distances between codes are not comparable: the jump from “No HS” to “HS grad” is not the same amount of education as “4-year” to “post-grad,” even though both are one code apart.
- c. **birthyr** — **interval, discrete in practice**. Differences are meaningful (someone born in 1952 is 21 years older than someone born in 1973), but the zero point (year 0) is arbitrary, so ratios are not: a person born in 1990 is not “twice” someone born in the year 995. Recorded in whole years, so discrete. Note that *age* derived from it (Question 6) does have a meaningful zero and is ratio-level.
- d. **Polity** — **ordinal by construction, often treated as interval**. The score is built by adding up coded components, and nothing guarantees that a move from -5 to 0 means the same amount of democratization as a move from $+5$ to $+10$. Much published work nevertheless treats it as interval; that is a substantive assumption the analyst makes, not a property of the measure. Either answer is defensible *with* this reasoning.
- e. **Vote share** — **interval-ratio, continuous**. A share of 0 means zero votes (a real zero), and 60% is meaningfully twice 30% . It is bounded between 0 and 100 , but boundedness does not change the level of measurement.
- f. **Coup indicator** — **binary, hence nominal and discrete**. Binary variables are the workhorse exception to “never average a nominal variable”: the average of a $0/1$ coup indicator is the *proportion* of country-years with a coup, which is perfectly meaningful. (Also worth noticing: the unit here is the country-year, not the country.)

Question 2: Units of analysis

- The evidence is about **states** — aggregate units. Each observation is a state with a median income and a statewide winner.
- The claim is about **individuals** — “rich Americans.”
- This is the **ecological fallacy**: inferring individual-level relationships from aggregate-level ones. Both patterns can hold at once because rich *states* and rich *people* are different things. Within every state, higher-income individuals can lean Republican, while the states with the highest median incomes (Connecticut, Massachusetts, ...) differ from Mississippi or West Virginia in many other ways — urbanization, education, religiosity, industry — that also move votes. The aggregate correlation mixes the individual income–vote relationship with everything else that varies across states. This is precisely the paradox documented in Gelman et al.’s *Red State, Blue State, Rich State, Poor State* (2008): the income–Republican gradient is strong *within* states (especially poor ones) even though richer states vote more Democratic.
- Individual-level data**: a survey (or validated voter-file data) recording each person’s income and vote choice. Only data at the unit of the claim can evaluate the claim.

Question 3: Operationalizing “voted”

- Self-report** — *strength*: cheap, available for anyone you can survey, and askable alongside attitudes and demographics. *Weakness*: people misreport, and not at random — nonvoters claim to have voted far more often than voters deny it. **Validated record** — *strength*: measures the actual behavior from an administrative record rather than memory or self-presentation. *Weakness*: it requires matching survey respondents to voter files, and matches fail — because of moves, name changes, clerical error, or registration gaps — so some true voters are coded as nonvoters.
- Systematic error**. Overreporting is driven by social desirability — voting is a civic norm, so the errors run overwhelmingly in one direction. A turnout rate computed from self-reports is therefore pushed **upward**, and no amount of extra respondents fixes it: unlike random error, systematic error does not average out.
- Lean on the **validated record** — self-report error is worst exactly where turnout is politically interesting. But validation has its own trap for this group: young people move often and register late, so voter-file matching fails for them at higher rates, and failed matches get coded as nonvoters. A “validated” youth turnout rate can therefore *understate* the truth (Ansolabehere & Hersh 2012). The methodological moral: swapping a bad measure for a better one does not exempt the better one from scrutiny.

Part B — Notation

Question 4: Summation

- (a) Adding a constant a to itself n times, and pulling constants through the summation sign:

$$(i) \sum_{i=1}^n a = na \quad (ii) \sum_{i=1}^n (a + bX_i) = na + b \sum_{i=1}^n X_i \quad (iii) \sum_{i=1}^4 (2i - 1) = 1 + 3 + 5 + 7 = 16$$

For (iii) you can also use the rules: $2 \sum_{i=1}^4 i - \sum_{i=1}^4 1 = 2(10) - 4 = 16$.

(b) With $X = (76, 73, 71, 65, 50)$:

$$\bar{X} = \frac{76 + 73 + 71 + 65 + 50}{5} = \frac{335}{5} = 67$$

State	X_i	$X_i - \bar{X}$
Minnesota	76	+9
Wisconsin	73	+6
New Hampshire	71	+4
Ohio	65	-2
Texas	50	-17

Sum of deviations: $9 + 6 + 4 - 2 - 17 = 0$.

(c) Split the sum and use the definition of $\bar{X}$:

$$\begin{aligned}\sum_{i=1}^n (X_i - \bar{X}) &= \sum_{i=1}^n X_i - \sum_{i=1}^n \bar{X} = \sum_{i=1}^n X_i - n\bar{X} \\ &= \sum_{i=1}^n X_i - n \cdot \frac{1}{n} \sum_{i=1}^n X_i = 0\end{aligned}$$

The average is defined so that the deviations cancel exactly — a fact we will lean on repeatedly from Week 2 onward.

(d) **False.** Smallest counterexample: $n = 2$ with $X = (1, 2)$:

$$\left(\sum_i X_i\right)^2 = (1 + 2)^2 = 9 \quad \text{but} \quad \sum_i X_i^2 = 1 + 4 = 5.$$

Squaring a sum produces cross-terms: $(\sum_i X_i)^2 = \sum_i X_i^2 + 2 \sum_{i < j} X_i X_j$, and the two sides agree only when the cross-terms happen to vanish. “The sum of squares is not the square of the sum” is worth tattooing somewhere before we meet variances.

Part C — Applied: the VOTER data

```
library(tidyverse)
# Path is relative to this file's folder in the course repository;
# adjust to wherever the CSV lives in your own project.
voter <- read_csv("../data/voter2012.csv")
```

Question 5: First contact

(a) Rows, columns, and the unit of observation:

```
dim(voter)
```

```
[1] 8000    9
```

8,000 rows and 9 columns. The unit of observation is the **individual survey respondent** — each row is one person interviewed in the 2012 VOTER Study.

(b) One `count()` call does it:

```
voter |> count(pid3, gender)
```

```
# A tibble: 8 x 3
  pid3 gender     n
  <dbl> <dbl> <int>
1     1     1  1796
2     1     2  1843
3     2     1  1559
4     2     2  1532
5     3     1   587
6     3     2   594
7    NA     1    44
8    NA     2    45
```

A more readable version, with codebook labels attached and genders spread into columns (the raw table above is a complete answer):

```
voter |>
  mutate(
    party = case_match(pid3, 1 ~ "Democrat", 2 ~ "Republican", 3 ~ "Other"),
    gender = case_match(gender, 1 ~ "Man", 2 ~ "Woman")
  ) |>
  count(party, gender) |>
  pivot_wider(names_from = gender, values_from = n)
```

```
# A tibble: 4 x 3
  party      Man Woman
  <chr>    <int> <int>
1 Democrat   1796  1843
2 Other      587   594
3 Republican 1559  1532
4 <NA>       44    45
```

(c) The most common combination is **Democratic women: 1,843 respondents** (code `pid3 = 1`, `gender = 2`), just ahead of Democratic men (1,796).

(d) `count()` reports NA as its own row: **89 respondents** are missing `pid3` (44 men and 45 women).

Question 6: How old is the electorate?

(a) Age is fully determined by birth year once you know the survey year — the survey was fielded in 2012, so age is just `2012 - birthyr`. Storing birth year rather than age is common in survey

data because birth year stays correct forever, while a stored “age” is only right in the year it was computed.

```
voter <- voter |> mutate(age = 2012 - birthyr)
```

(b) The youngest and oldest ages, and how many respondents have each:

```
voter |> summarise(youngest = min(age), oldest = max(age))
```

```
# A tibble: 1 x 2
  youngest oldest
    <dbl>   <dbl>
1      18     91
```

```
voter |>
  count(age) |>
  filter(age == min(age) | age == max(age))
```

```
# A tibble: 2 x 2
  age      n
  <dbl> <int>
1    18     5
2    91     2
```

The youngest respondents are **18** (born 1994) — there are **5** of them — and the oldest are **91** (born 1921), of whom there are **2**. Both endpoints are sensible for a 2012 voter survey: 18 is the voting age, and nobody claims to be 130, which is itself a useful data check.

(c) `2012 - birthyr` is a respondent’s exact age only if their birthday had already passed on the interview date; otherwise it overstates age by one year. For describing 8,000 respondents whose ages span 18–91, an off-by-one that affects part of the sample is harmless — but it is the kind of assumption worth *noticing* out loud, which is why we asked.

Question 7 (stretch): Something is off

Many findings work here; the three biggest hazards are below, each visible from a single `count()` with the codebook open. (Week 2’s lab is where we repair all of them.)

Hazard 1: 5 means “Don’t know,” not “more than Strongly disagree.”

```
voter |> count(important_econ)
```

```
# A tibble: 6 x 2
  important_econ      n
    <dbl>   <int>
1          1  6983
2          2   823
3          3    45
4          4    13
5          5    39
6         NA    97
```

The codebook says code 5 on the four issue items is *Don't know* — a nonresponse hiding inside the scale. Any arithmetic that treats it as a number is contaminated:

```
mean(voter$important_econ, na.rm = TRUE)    # wrong: DK counted as 5
```

```
[1] 1.1402
```

```
voter |>
  mutate(econ = na_if(important_econ, 5)) |>
  summarise(mean_econ = mean(econ, na.rm = TRUE))    # DK as missing
```

```
# A tibble: 1 x 1
  mean_econ
    <dbl>
1      1.12
```

The damage is small here (1.14 vs. 1.12) because only 39 people said Don't know — but nothing in the output warns you, and on a different item the gap could be enormous. The fix (`na_if()`) must come from reading the codebook, not from staring at the numbers.

Hazard 2: two items are reverse-framed.

```
voter |> count(unimportant_budget)
```

```
# A tibble: 6 x 2
  unimportant_budget      n
    <dbl> <int>
1             1      193
2             2      659
3             3     1993
4             4     5027
5             5        39
6            NA        89
```

At first glance the budget deficit looks like nobody's priority — the largest group (5,027 people) is at code 4. But the question reads “the budget deficit **is not** an important issue,” so code 4 (*Strongly disagree*) means those 5,027 respondents think the deficit **is** important. Compare raw codes on `important_econ` and `unimportant_budget` and every conclusion comes out backwards; the reverse-framed items must be flipped before the four issue items can be compared or combined.

Hazard 3: `pol_interest` hides its nonresponse as a high code.

```
voter |> count(pol_interest)
```

```
# A tibble: 5 x 2
  pol_interest      n
    <dbl> <int>
1             1     509
2             2    2359
3             3    5063
4             4      57
5            NA     12
```

The scale runs from 1 (*Not much*) up to 3 (*Very much*) — and then code 4 is *Not sure*. Numerically, 57 “Not sure” respondents sit *above* “Very much,” so any average or sort on the raw codes quietly treats maximal uncertainty as maximal interest.

Also acceptable: the genuinely blank cells (`read_csv()` turns them into `NA` — e.g., the 89 missing `pid3` from Question 5), or the bare-integer coding of every variable, which makes the codebook the only source of meaning. The common thread, and the habit this course will drill: **the numbers alone never tell you what the numbers mean — read the codebook first.**